Automated E-Commerce Deployment Platform

Project Documentation: Part 1 Project Planning & Management | Part 2 Literature Review | Part 3 Requirements Gathering

Digital Egypt Pioneers Initiative (DEPI) - Graduation Project | Version 1.0 | 19 October 2026

GitHub Repository: https://github.com/zeyad-mohammed-dev/Automated-E-Commerce-Deployment-Platform

Team Member	Role
Zeyad Mohammed	Team Leader
Serag Mohammed	Team Member
Ali Fathy	Team Member
Ahmed Gomaa	Team Member

Part 1: Project Planning & Management

1.1 Project Proposal

Overview. Automated E-Commerce Deployment Platform is a DevOps project that takes a multi-service e-commerce application (web frontend, backend API, payment service, search service and a database) and delivers it through a fully automated pipeline. Every code change is built, tested, packaged into containers and deployed to a Kubernetes cluster on AWS, and the infrastructure itself is created from code. The focus is the delivery platform (automation, scaling, monitoring and reliability), not a feature-rich online store.

Problem statement. E-commerce websites often suffer from downtime and complicated deployments. Manual releases are slow and error-prone, a faulty release can take the store offline, and traffic spikes can overwhelm a fixed number of servers.

Objectives.

1. Containerize every service with Docker so it behaves identically in development, testing and production.
2. Automate build, test and deployment with GitHub Actions pipelines triggered by code changes.
3. Provision AWS infrastructure (EC2, RDS, S3, ELB) reproducibly with Terraform, and configure servers with Ansible.
4. Deploy on Kubernetes with automatic scaling (Horizontal Pod Autoscaler) to absorb load changes.
5. Route traffic through an Nginx reverse proxy behind an AWS Elastic Load Balancer.
6. Monitor services with Prometheus and Grafana dashboards and alerts.
7. Provide automated rollback so a failed release is reverted without manual work.
8. Keep all code, pipelines and infrastructure scripts versioned in a single Git repository.

Scope.

- In scope: a multi-service e-commerce application (kept intentionally simple), Docker images, CI/CD pipelines, Kubernetes deployment with auto-scaling, Terraform and Ansible automation, Nginx routing, Prometheus/Grafana monitoring, automated rollback, testing and documentation.
- Out of scope: advanced store features (recommendations, reviews, loyalty), processing real payments or storing real card data, multi-region deployment, mobile applications.

Assumptions and constraints.

- The project must be delivered within the DEPI schedule (final presentation and reports due 4 December 2026).
- AWS budget is limited, so the environment is created on demand with Terraform and destroyed after testing and demos.
- Frontend and backend technologies, and the search implementation, are selected by the team during the System Analysis & Design phase.

- The project brief names Jenkins as an example CI/CD tool. The team implements the same CI/CD requirement with GitHub Actions, because the code already lives on GitHub and no separate CI server must be maintained.

Deliverables.

#	Deliverable	Description
D1	Dockerized microservices	Dockerfiles and images for frontend, backend, payment and search services
D2	CI/CD pipelines	GitHub Actions workflows: build, test, image publish, deploy
D3	Kubernetes deployment	Manifests with health probes, rolling updates and auto-scaling
D4	Ansible scripts	Server and configuration setup
D5	Monitoring	Prometheus metrics, alerts and Grafana dashboards
D6	Nginx reverse proxy	Request routing to services
D7	Terraform scripts	AWS EC2, RDS, S3 and ELB
D8	Automated rollback	Rollback strategy triggered by failed deployments
D9	Git repository and documentation	Code, scripts, README, design, test and user documents

1.2 Project Plan

Phases and timeline. Dates for later phases follow the DEPI Project Documentation Guidelines.

Phase	Period	Main outputs	DEPI deadline
1. Initiation & Planning	up to 19 Oct 2026	Project Planning, Literature Review, Requirements Gathering	19 Oct 2026
2. System Analysis & Design	20 Oct - 6 Nov 2026	UML diagrams, ERD, architecture, technology stack, environment setup	6 Nov 2026
3. Implementation	7 Nov - 30 Nov 2026 (development starts earlier)	Source code, pipelines, infrastructure scripts, working deployment	30 Nov 2026
4. Testing, Final Reports & Presentation	23 Nov - 4 Dec 2026	Test reports, user manual, technical documentation, presentation, infographic	4 Dec 2026

Gantt chart (each column is the week starting on the date shown; filled cell = planned work).

Task	19 Oct	26 Oct	2 Nov	9 Nov	16 Nov	23 Nov	30 Nov
Planning documents (Parts 1-3)	■						
Analysis & design documents	■	■	■				
Environment setup (AWS account, repo structure, branching, workflow skeleton)		■	■				
Application services development			■	■	■		
Dockerization			■	■			
Terraform infrastructure			■	■	■		
Ansible configuration				■	■		
CI/CD pipelines			■	■	■		
Kubernetes deployment & auto-scaling				■	■	■	
Nginx routing & load balancer					■	■	
Monitoring & alerts					■	■	
Automated rollback						■	
Testing & QA					■	■	■
Final documents, presentation, demo, infographic						■	■

Milestones.

Milestone	Target date	Completion criterion
M1 Planning documents submitted	19 Oct 2026	PDF uploaded to GitHub and submitted on the LMS
M2 Design submitted	6 Nov 2026	All required analysis and design documents uploaded
M3 Containers and CI ready	14 Nov 2026	All services build as images; pipeline runs on every push
M4 Running on AWS	21 Nov 2026	Terraform and Ansible create the environment; application reachable through the load balancer
M5 Implementation	30 Nov 2026	Auto-scaling, monitoring and rollback

Milestone	Target date	Completion criterion
submitted		demonstrated; code and README complete
M6 Final delivery	4 Dec 2026	Test reports, manuals, presentation and infographic submitted

Resource allocation.

- People: four team members working part-time alongside the DEPI program; ownership areas are defined in section 1.3.
- Tools: GitHub (repository, Actions, Projects board), Docker, Kubernetes, Terraform, Ansible, Prometheus, Grafana, Nginx.
- Cloud: AWS EC2, RDS, S3 and ELB, created on demand and destroyed after use to control cost.

1.3 Task Assignment & Roles

The work is divided into ten responsibility areas. Each member will be primary owner of two to three areas and backup for others, so no area depends on a single person. The Team Leader also coordinates planning and submissions. Every change is reviewed by at least one other member through a pull request.

ID	Area	Responsibilities	Main deliverables	Primary owner	Backup
R1	Project management & documentation	Planning, task board, meetings, DEPI/LMS submissions, documents, final presentation	Parts 1-3, design and final documents, presentation	Zeyad Mohammed	Serag Mohammed
R2	Frontend	Storefront and admin interface, calls to the backend API	Containerized frontend	Zeyad Mohammed	Ali Fathy
R3	Backend & services	Catalog, users, cart and order APIs, payment service (test mode), search service, database schema	Containerized services, API documentation	Zeyad Mohammed	Ali Fathy
R4	Containerization (Docker)	Dockerfiles, optimized images, local multi-service environment, image registry	D1	Zeyad Mohammed	Ali Fathy
R5	CI/CD (GitHub Actions)	Build, test, image and deploy workflows, branch protection, secrets handling, rollback trigger	D2	Ahmed Gomaa	Zeyad Mohammed

ID	Area	Responsibilities	Main deliverables	Primary owner	Backup
R6	Infrastructure as Code (Terraform)	Networking, EC2, RDS, S3, ELB, access policies, state management	D7	Serag Mohammed	Ali Fathy
R7	Configuration management (Ansible)	Server setup and hardening, Kubernetes prerequisites, Nginx configuration	D4, part of D6	Serag Mohammed	Zeyad Mohammed
R8	Kubernetes	Deployments, services, probes, configuration and secrets, rolling updates, auto-scaling, traffic routing with Nginx	D3, part of D6	Ahmed Gomaa	Ali Fathy
R9	Monitoring & observability	Prometheus metrics, alert rules, Grafana dashboards	D5	Ali Fathy	Ahmed Gomaa
R10	Testing & QA	Test plan, unit and integration tests, load tests for scaling, rollback drills, bug reports	D8 validation, test reports	Ali Fathy	Zeyad Mohammed

Working agreements. Feature branches merged by pull request into a protected main branch; a GitHub Projects board for tasks; a weekly team meeting plus short written updates; no credentials in the repository (secrets stored in GitHub and AWS secret mechanisms).

1.4 Risk Assessment & Mitigation Plan

ID	Risk	Likelihood	Impact	Mitigation
K1	AWS costs exceed the budget	Medium	High	Small instance sizes, budget alerts, create the environment only when needed and destroy it with Terraform afterwards
K2	Short implementation window	High	High	Keep the application simple, start infrastructure work during design, track milestones weekly
K3	Skill gaps in Kubernetes, Terraform or AWS	Medium	Medium	Task pairing, backup owners, official documentation, early proof-of-concept
K4	Credentials leaked in the repository	Low	High	Secrets in GitHub Secrets, secret scanning, review checklist, rotate keys if exposed
K5	Bad release causes downtime	Medium	High	Health probes, rolling updates, automated rollback, deployment smoke tests

ID	Risk	Likelihood	Impact	Mitigation
K6	Database data loss	Low	High	RDS automated backups, separate infrastructure code for data resources, restore test
K7	Integration conflicts between members	Medium	Medium	Feature branches, pull-request reviews, agreed service interfaces
K8	Team member unavailable	Medium	Medium	Backup owner for every area, documented procedures in the repository
K9	AWS limits or access problems	Low	Medium	Check account and service limits early, keep a local Kubernetes fallback for development
K10	Payment feature misunderstood as real payments	Low	Medium	Use test mode or mock provider only, never store card data, state this in scope

1.5 Key Performance Indicators (KPIs)

The targets are initial goals and will be validated during testing.

KPI	Target	How it is measured
Deployment frequency	Several deployments per week during implementation	Count of successful pipeline deploy runs
Pipeline duration (commit to deployed)	Under 15 minutes	GitHub Actions run times
Change failure rate	15% or lower	Failed deployments divided by total deployments
Automated rollback time	Under 5 minutes after a failed deployment	Timestamps in pipeline and Kubernetes events
System uptime during test period	99% or higher	Prometheus availability metrics
Response time (95th percentile)	Under 500 ms at the planned test load	Load test results and Prometheus metrics
Auto-scaling reaction	Additional replicas running within about 2 minutes of sustained high load	Kubernetes scaling events during load test
Environment rebuild time	Full environment from code in under 45 minutes	Timed Terraform plus Ansible run

KPI	Target	How it is measured
Automated test pass rate	100% required before deployment	Pipeline test stage
Milestone completion	All six milestones met on time	Milestone table review

Part 2: Literature Review

2.1 Purpose and Method

This review looks at open-source reference e-commerce platforms, the tools that could be used for each part of the delivery pipeline, and published research on measuring software delivery. The goal is to learn from existing solutions and to show how our project differs. Sources are listed in the References section.

2.2 Similar Projects

Google Online Boutique (microservices-demo). A web e-commerce application where users browse items, add them to a cart and purchase them. It consists of 11 microservices written in different languages that communicate over gRPC, and it includes a Locust-based load generator that simulates realistic traffic. Google uses it to demonstrate Kubernetes/GKE, Istio and related technologies, and it runs on any Kubernetes cluster. Later releases added deployment variations with Kustomize and a Terraform option.

- Strengths: realistic multi-service design, built-in load generation, many documented deployment variations, active releases.
- Limitations for our case: it is a technology showcase, so its deployment examples center on Google Cloud. An AWS-based pipeline with automated rollback and monitoring alerts is something our team must design ourselves.

Sock Shop (Weaveworks and Container Solutions). A demo application simulating the user-facing part of a shop that sells socks, built with Spring Boot, Go kit and Node.js and packaged in Docker containers. It ships deployment scripts for several platforms, including Docker, Kubernetes and Amazon ECS.

- Strengths: polyglot microservices, many deployment targets, widely used for demos and testing.
- Limitations: the main repository is archived (December 2023) and marked deprecated, so it is no longer maintained and its stack ages.

Traditional manual deployment (common practice). Teams deploy to one or a few servers by hand or with ad-hoc scripts.

- Strengths: simple to start, low initial cost.
- Limitations: slow and error-prone releases, downtime during updates, no automatic scaling, difficult to reproduce environments.

2.3 Technology Options Considered

Area	Options considered	Our choice	Rationale

Area	Options considered	Our choice	Rationale
CI/CD	Jenkins (self-hosted server), GitHub Actions (workflows stored in the repository)	GitHub Actions	The repository is already on GitHub; workflows live next to the code; no CI server to install and maintain
Runtime and scaling	Containers on a single server, Kubernetes	Kubernetes	Self-healing, rolling updates, rollback and Horizontal Pod Autoscaler; the autoscaler adjusts the number of Pods using metrics such as CPU or memory usage, checking on a periodic control loop (15 seconds by default)
Provisioning vs configuration	Terraform, Ansible	Both, complementary	Terraform creates AWS resources declaratively; Ansible configures the servers created
Monitoring	Prometheus with Grafana	Prometheus with Grafana	Metrics collection with alert rules and dashboards, standard in Kubernetes environments
Traffic entry	Cloud load balancer, Nginx reverse proxy	AWS ELB plus Nginx	ELB spreads traffic across nodes; Nginx routes requests to the right service

2.4 Measuring Delivery Performance

The DORA research program identifies four key measures of software delivery: deployment frequency, lead time for changes, change failure rate and time to restore service. They cover both speed (throughput) and stability. Published guidance reports that high-performing teams keep the change failure rate roughly between 0% and 15%. Our KPIs (section 1.5) adopt these measures so the project's success can be judged against an accepted industry reference.

2.5 Comparison with Our Project

Aspect	Online Boutique	Sock Shop	Our platform
Primary purpose	Demonstrate cloud-native technologies	Demonstrate and test microservice technologies	Automate delivery, scaling, monitoring and rollback of an e-commerce platform
Architecture	11 microservices	Microservices in Spring Boot, Go	Frontend, backend, payment and search services with a database;

Aspect	Online Boutique	Sock Shop	Our platform
	over gRPC	kit, Node.js	technologies chosen in the design phase
Deployment targets	Any Kubernetes cluster, Google Cloud examples, Kustomize and Terraform variations	Docker, Kubernetes, Amazon ECS and others	Kubernetes on AWS (EC2, RDS, S3, ELB)
Maintenance status	Releases published (v0.4.x)	Archived, deprecated	Maintained by the team during the project
Our added focus			GitHub Actions pipelines, auto-scaling, Prometheus alerts, automated rollback, environment reproducible from Terraform and Ansible

2.6 Lessons Applied

- Include a load generator in testing (as Online Boutique does) to prove auto-scaling works.
- Keep deployment code in the repository next to the application, as both reference projects do.
- Prefer actively maintained tools and document every dependency, to avoid the aging problem seen in Sock Shop.
- Track DORA-style measures from the first pipeline run.
- Keep the application small so effort goes into the delivery platform.

2.7 Conclusion

Existing projects show how to structure a microservice store and deploy it on Kubernetes, but they are demonstrations rather than complete, AWS-based delivery platforms. Our project combines their proven ideas with GitHub Actions automation, infrastructure as code, auto-scaling, monitoring and automated rollback.

Part 3: Requirements Gathering

3.1 Stakeholder Analysis

Stakeholder	Description	Needs and expectations
Customers	Shoppers using the storefront	Fast, always-available store; simple browsing, search and checkout; safe payments
Store administrator	Manages products and orders	Easy product and order management; reliable system

Stakeholder	Description	Needs and expectations
DevOps team (our team)	Builds and operates the platform	Automated releases, clear monitoring, quick rollback, reproducible environments
Developers	Contribute application code	Fast feedback from automated tests, simple release process
DEPI instructors and evaluators	Assess the graduation project	Complete documentation, working demo, evidence of DevOps practices
Payment provider (external)	Test or mock payment service	Correct API usage; no real card data handled

3.2 User Stories

ID	User story
US-01	As a customer, I want to register and log in so that I can place and track orders.
US-02	As a customer, I want to browse products so that I can find what I need.
US-03	As a customer, I want to search for products so that I can find items quickly.
US-04	As a customer, I want to add items to a cart so that I can buy several products together.
US-05	As a customer, I want to pay at checkout so that my order is confirmed.
US-06	As a customer, I want to view my order history so that I can follow my purchases.
US-07	As an administrator, I want to add and update products so that the catalog stays current.
US-08	As an administrator, I want to view and manage orders so that I can fulfil them.
US-09	As a developer, I want my change tested and built automatically after each push so that I get fast feedback.
US-10	As a DevOps engineer, I want deployments to run automatically so that releases are quick and consistent.
US-11	As a DevOps engineer, I want a failed release rolled back automatically so that customers see no prolonged downtime.
US-12	As a DevOps engineer, I want dashboards and alerts so that I notice problems before customers do.
US-13	As a DevOps engineer, I want to create the whole environment from code so that it is repeatable.

3.3 Use Cases

ID	Use case	Actor	Summary
UC-01	Register and log in	Customer	Create an account and sign in
UC-02	Browse and search products	Customer	View the catalog and search it
UC-03	Manage cart	Customer	Add, change or remove items
UC-04	Checkout and pay	Customer, Payment provider	Place an order and complete payment in test mode
UC-05	View orders	Customer	See past and current orders
UC-06	Manage catalog and orders	Administrator	Maintain products; process orders
UC-07	Push code change	Developer	Commit triggers build and tests
UC-08	Deploy release	DevOps engineer	Pipeline publishes images and updates the cluster
UC-09	Roll back release	DevOps engineer, Pipeline	Previous version restored after a failed deployment
UC-10	Monitor and alert	DevOps engineer	View dashboards; receive alerts
UC-11	Provision environment	DevOps engineer	Terraform and Ansible create the AWS environment

3.4 Functional Requirements

A. Application

ID	Requirement
FR-A01	Users can register, log in and log out.
FR-A02	The system displays a product catalog with details.
FR-A03	Users can search products through a search service.
FR-A04	Users can manage a shopping cart.
FR-A05	Users can place orders and see their order history.
FR-A06	The payment service processes payments in test or mock mode only.
FR-A07	Administrators can create, update and remove products and view orders.
FR-A08	Product, user and order data are stored in a database (AWS RDS in deployment).

ID	Requirement
FR-A09	Product images and static files are stored in object storage (AWS S3).

B. Delivery platform

ID	Requirement
FR-P01	Every service runs as a Docker container built from a Dockerfile in the repository.
FR-P02	A GitHub Actions pipeline builds and tests the code on every pull request and push.
FR-P03	On merge to the main branch, the pipeline builds images, publishes them to a registry and deploys to Kubernetes.
FR-P04	Kubernetes Deployments use health probes and rolling updates.
FR-P05	Services scale automatically with load through the Horizontal Pod Autoscaler.
FR-P06	Nginx reverse proxy routes requests to the correct service; an AWS ELB receives external traffic.
FR-P07	Terraform creates AWS resources (EC2, RDS, S3, ELB and networking).
FR-P08	Ansible configures servers and installs required software.
FR-P09	Prometheus collects service and container metrics; Grafana shows dashboards.
FR-P10	Alert rules notify the team about failures or high resource use.
FR-P11	A failed deployment is detected and rolled back automatically to the last working version.
FR-P12	All code, pipelines, manifests and scripts are stored in one Git repository with a documented branching workflow.

3.5 Non-Functional Requirements

ID	Category	Requirement
NFR-01	Performance	95th percentile response time under 500 ms at the planned test load
NFR-02	Scalability	Each service can run multiple replicas and scale up and down automatically
NFR-03	Availability	Uptime of 99% or higher during the test period; no downtime during normal rolling updates
NFR-04	Reliability	Failed releases are reverted automatically; database backups are enabled

ID	Category	Requirement
NFR-05	Security	No credentials in the repository; secrets in secure stores; least-privilege access; HTTPS for external traffic; no real card data handled
NFR-06	Maintainability	Modular services, consistent naming, documented README and runbooks
NFR-07	Reproducibility	The whole environment can be rebuilt from code in under 45 minutes
NFR-08	Observability	Every service exposes health and metrics endpoints
NFR-09	Usability	The storefront is simple, responsive and usable on common browsers and devices
NFR-10	Portability	Services run the same in local Docker and on the cloud cluster
NFR-11	Cost efficiency	Resources are sized small and removed when not in use

References

1. Google Cloud Platform. Online Boutique (microservices-demo) repository and release notes. https://github.com/GoogleCloudPlatform/microservices-demo (accessed 19 Oct 2026).
2. Weaveworks and Container Solutions. Sock Shop: A Microservice Demo Application. https://github.com/microservices-demo/microservices-demo (accessed 19 Oct 2026).
3. Kubernetes Documentation. Horizontal Pod Autoscaling. https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/ (accessed 19 Oct 2026).
4. DORA. Software delivery metrics: the four keys. https://dora.dev/guides/dora-metrics-four-keys (accessed 19 Oct 2026).
5. Codefresh. DORA Metrics: 4 Key Metrics for Improving DevOps Performance. https://codefresh.io/learn/software-deployment/dora-metrics-4-key-metrics-for-improving-devops-performance (accessed 19 Oct 2026).
6. GitHub Docs. GitHub Actions. https://docs.github.com/en/actions
7. HashiCorp. Terraform Documentation. https://developer.hashicorp.com/terraform/docs
8. Ansible Documentation. https://docs.ansible.com/
9. Prometheus Documentation. https://prometheus.io/docs/introduction/overview/
10. Jenkins Documentation. https://www.jenkins.io/doc/